

Exploring GPU-Based Workload Scheduling Techniques for Edge Computing

Michail Tsenos

Department of Informatics

Athens University of Economics And Business

Athens, Greece

tsemike@aueb.gr

Vana Kalogeraki

Department of Informatics

Athens University of Economics And Business

Athens, Greece

vana@aueb.gr

Abstract—This paper addresses the challenges of hosting multiple GPU workloads in resource-constrained edge environments where GPU computation demands often exceeds availability. Traditional approaches to GPU sharing, such as NVIDIA MPS and MIG, have limitations regarding dynamic priority adjustments and hardware compatibility, as several of them require specialized data-center oriented GPUs. To overcome these limitations, we propose a novel, hardware-agnostic GPU spatio-temporal sharing and scheduling mechanism for GPU-intensive, long-running jobs in shared edge clusters requiring non-to-minimum changes in the workload source code. A Local GPU Scheduler Component, can prioritize the execution of each workload based on a set of scheduling policies, including Priority Based Scheduling and Real-Time, Laxity based scheduling in order to achieve the SLOs of time-critical real-time workloads that runs concurrently in the shared GPU. Extensive evaluation on a set of non specialized GPUs with different architectures provides fully insight on the efficiency and the efficacy of the proposed technique.

Index Terms—Scheduling, GPU, Edge Computing

I. INTRODUCTION

Graphic Processing Units, or more commonly known as Graphic Cards or GPUs, have become a cornerstone of modern computing. Originally designed to accelerate rendering in gaming and graphics, GPUs now power a wide range of applications, from artificial intelligence (AI) and machine learning (ML) to scientific simulations, financial modeling, and video processing to more advanced scenarios like ML Pipelines for automated data extraction from handwritten texts and published studies. For example, a GPU-accelerated ML pipeline can extract medicinal plant data from ethnobotanical research papers and handwritten folklore manuscripts with OCR and NLP algorithms, and then field experts can use predictive and generative GPU-accelerated ML models on the extracted data for the discovery of new drugs.

With their capacity to execute thousands of calculations in parallel, GPUs have now become essential for high-performance computing (HPC) tasks, which makes them indispensable in both consumer and enterprise-level applications. However, the increasing demand of GPU resources often exceeds the availability of resources in organizations or data-centers. An increasing number of organizations migrate their GPU accelerated workloads from the cloud to edge private clusters, as many companies find it more cost-effective to buy

GPUs on premise than to rent on the cloud. A second important reason is privacy. Many workloads, especially machine learning workloads, handle potentially sensitive or private data (*i.e.*, medical information) that a lot of companies are not allowed to share outside of their corporate network or cannot be shared due to the European Union GDPR rules.

Hosting multiple GPU workloads on edge clusters presents several challenges due to the constraints of edge environments and the complexities of GPU resource management. In contrast to centralized cloud data centers, edge clusters typically have fewer resources, resulting in small computational capacity. Furthermore, they have limited power and cooling capabilities. As a result, it is difficult to horizontally increase the cluster capacity by adding more physical resources that demand even more power and cooling. Managing multiple GPU workloads in such settings requires efficient allocation of resources to ensure computational performance, as competing workloads can lead to contention, degraded performance, and oversubscription of the GPUs. Furthermore, edge clusters often face dynamic workloads and varying latency requirements, making it difficult to optimize GPU usage for diverse applications such as deep learning training, video processing, or data analytics. The lack of standardized tools for scheduling and orchestrating workloads across diverse GPUs, varying in resources or even manufacturer (*e.g.*, NVIDIA, AMD), combined with the hardware variability among edge nodes, adds complexity and makes efficient workload hosting a significant challenge.

Resource sharing across different workloads is typically performed by employing the underlying NVIDIA tools for GPU multiprocessing. The NVIDIA Multi-Process Service (MPS) [1] is a feature designed to enhance the efficiency and performance of GPU resources when running multiple parallel workloads. MPS allows multiple applications or processes to share a single GPU while maintaining high performance and resource isolation. By enabling concurrent kernel execution, MPS optimizes GPU utilization, reducing idle time and ensuring that the computational capacity is used effectively. This is particularly beneficial in scenarios involving many small or medium-sized workloads, as it reduces the overhead associated with context switching and allows for better resource allocation. MPS is widely used in applications such as AI inference, data processing, and high-performance computing,

where efficient GPU sharing is critical to meet the demands of multiple users or services running on shared infrastructure. Although the user can set a priority for each workload, that priority is permanent and cannot *change dynamically* during the execution of the workload without restarting the workload process, which means that the workload cannot use more than the predefined allocated resources in situations that the GPU is shared with other workloads. Time Slicing is the simplest version of MPS, where all the processes have the same priority and the "time-slices" of the GPU are equally shared among the workloads [1], [2]. On newer GPU architectures, post-Volta architectures, NVIDIA MPS benefits from advanced GPU features such as thread block-level preemption, independent SM (Streaming Multiprocessor) scheduling, and faster context switching, which enable more efficient sharing of resources among multiple CUDA processes.

In parallel, NVIDIA Multi-Instance GPU (MIG) technology [3] is a feature that enables a single GPU to be partitioned into multiple isolated instances, each with its own dedicated resources. MIG allows each instance to allocate a portion of the GPU's memory, cores, and bandwidth, ensuring predictable performance for diverse workloads running simultaneously. This technology is especially useful in multi-tenant or shared environments, like edge clusters, where applications with varying computational needs race for shared resources. MIG is ideal for workloads like AI inference, data analytics, enabling organizations to maximize GPU utilization and scalability. Although MIG is cutting-edge technology, not all GPU models support it. MIG is supported on GPUs starting with the NVIDIA Ampere generation. This restricts its adoption in environments with older GPU models, which remain prevalent in many organizations due to cost constraints or legacy system dependencies. Additionally, not all Ampere-based GPUs support MIG; for instance, certain consumer and lower-tier GPUs lack this capability, further limiting the technology's reach. In general, only datacenter oriented NVIDIA GPUs, support this feature. MIG can lead to underutilization of resources if workloads do not fully leverage their allocated GPU instance. The static allocation of resources may not suit dynamic or bursty workloads that require flexibility. Furthermore, the complexity of configuring and managing MIG instances may pose a challenge for organizations without specialized expertise in GPU resource management, especially in heterogeneous environments.

In this work we make the following contributions:

- We provide a generic mechanism that enables heterogeneous workloads to dynamically share GPU resources on GPUs that do not natively support virtualization.
- We provide a scheduling interface for implementing hardware-agnostic scheduling policies for GPU enabled workloads.
- Our approach integrates very easily on existing code bases with no to minimal changes and is compatible with most frameworks and tools such as Tensorflow [4], PyTorch [5], and FFMpeg.
- Our extensive evaluation illustrates that our approach

proved to perform significantly better for dynamic workloads on oversubscribed GPUs.

II. BACKGROUND ON GPU SCHEDULING

Graphics Processing Units (GPUs) are specialized hardware designed for high-performance parallel computation. Initially, GPUs were developed for graphics rendering but, due to their high performance, now they are widely used in fields like artificial intelligence, scientific simulations, and data analytics. Unlike CPUs, which have a few powerful cores optimized for sequential tasks, GPUs consist of thousands of smaller, efficient cores organized into streaming multiprocessors (SMs) and designed for high level of parallelism. They also contain various types of compute cores such as fp64 units and tensor cores, register files, and L1 cache.

Threads are grouped into warps and execute instructions simultaneously. Applications leverage GPUs by decomposing workloads into *smaller tasks*, launching them as *kernels* with specified thread and block configurations, and relying on the GPU scheduler to manage execution across SMs. A kernel is a function that runs on the GPU in parallel. It allows the execution of code simultaneously across many threads of the GPU.

Typically, when a process needs to launch a new kernel, a system call is executed by the CPU and transferred to the GPU via the PCIe bus. For example, in NVIDIA GPUs, the CPU issues a kernel launch by calling a CUDA API function, which sends a command to the GPU driver. The driver queues the kernel for execution, and the GPU scheduler picks it up and runs it on the GPU's streaming multiprocessors.

Upon the execution of each kernel, the result is retrieved by the CPU and the cycle continues. When multiple processes are executed in parallel, the kernels from each process run can run simultaneously on the same GPU. In NVIDIA GPUs, the *Multi-Process Service (MPS)* enables more efficient sharing of a single GPU among multiple CUDA applications. It does this by intercepting kernel launches and allowing concurrent execution of kernels from different processes. This improves overall resource utilization. However, when multiple kernels run concurrently on the same GPU, they compete for compute resources, which can impact performance.

Machine learning frameworks such as Tensorflow or PyTorch offer high-level APIs that compile the user code into a sequence of kernel launches on the GPU. In NVIDIA cards, each kernel launch and memory operation on the GPU is mapped to a CUDA stream. A stream is a sequence of operations that are executed in order.

The GPU scheduler assigns thread blocks to streaming multiprocessors (SM) once their data dependencies are resolved and sufficient resources are available on the SM. Users cannot control the execution sequence of the thread blocks. The scheduler balances workloads, interleaves warp execution to hide memory latency, and dynamically allocates resources such as memory and compute cores. Modern GPUs also support asynchronous execution, enabling overlapping computation and communication for higher efficiency.

Summarizing, for each application, the CPU prepares the input data for the GPU by allocating memory, transferring the data from the system's main memory to the GPU's memory, usually via PCIe, and then launching the kernel with specified thread configurations (grid and block dimensions). Once the kernel is launched, the GPU scheduler manages the kernel execution by assigning thread blocks to the available streaming multiprocessors (SMs). The CPU can either wait for the GPU to complete execution through explicit synchronization (e.g., `cudaDeviceSynchronize`) or continue executing other tasks asynchronously. Once the GPU finishes, the CPU retrieves the results from the GPU memory for further processing.

III. PROPOSED ARCHITECTURE

In order to address the challenges of supporting long running heterogeneous GPU workloads on edge clusters on shared GPUs, we designed and implemented a scheduling component that is instantiated at each node of the cluster. We call this component Local Scheduler Component or LSC for short. LSC allows the user and system administrators to easily submit and manage GPU enabled workloads on GPUs that do not support virtualization, such as NVIDIA RTX series like RTX 4090 or even older, but still extensively used, cards like the GTX 1080Ti.

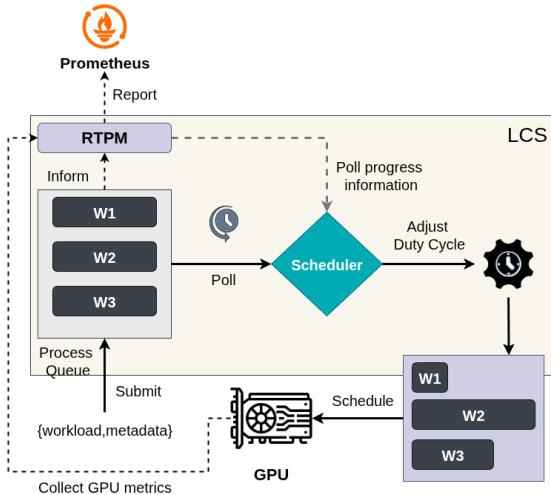


Fig. 1: Architecture overview of LCS

Each workload is submitted via the LSC and includes metadata that present the workload's associated demands: memory and priority or deadline. Memory is the amount of GPU memory that the workload needs in order to execute. If the required amount of GPU memory exceeds the available GPU memory, then, LSC, can either reject the workload, or place it back on the queue to wait for resources to become available. Priority and deadline are two metadata that are used by the scheduler to estimate the urgency of the workload. The scheduling decisions of each workload are affected by these two parameters, depending on the corresponding scheduling algorithm.

If sufficient resources are available, the workload, which can be packaged as a Docker container, is shipped and deployed to the node. In the case of workloads with real-time demands, LCS is responsible for tracking the progress of each workload and performing scheduling actions in order to maximize the probability that the workload achieves its deadline, as described in Section IV. The component of the LCS that is responsible for tracking the progress of each workload is the Real-Time Performance Monitor (RTPM). RTPM periodically receives *progressUpdates* from each workload, either by receiving *PROGRESS UPDATE* requests or by periodically monitoring the workload logs. RTPM also collects GPU metrics such as SM, encoder and decoder utilization and available memory via NVML [6]. The NVIDIA Management Library (NVML) is an API that provides low-level access to GPU hardware for monitoring and management tasks. It enables querying of GPU metrics such as temperature, utilization, memory usage, and power consumption, and is widely utilized in tools like `nvidia-smi` and data center monitoring systems. For cluster-wide observability, RTPM periodically publishes its metrics to a central Prometheus [7] server. Prometheus is a widely used monitoring system that allows fine-grained monitoring of collected metrics of large clusters. Figure 1 shows an overview of the LCS architecture.

At each node, in the case that the GPUs are manufactured by NVIDIA, an MPS server is running to allow more efficient GPU sharing across the workloads. We have to note here, that, we *do not* define the `CUDA_MPS_ACTIVE_THREAD_PERCENTAGE` of the workloads, allowing them to utilize all available GPU threads.

IV. METHODOLOGY

The objective of our work is to provide a simple mechanism that enables us for *dynamically* sharing resources among processes requesting GPUs that lack native virtualization support, such as NVIDIA MIG, while also allowing users to implement dynamic algorithms for scheduling heterogeneous workloads on these devices.

A. Existing solutions

While contemporary GPUs support concurrent execution of multiple processes, they offer limited or no support for implementing customizable scheduling policies at the user or system level. Many works in the literature [8] [9] [10] [11] [12] try to overcome this limitation with specialized methods that work in specific cases but cannot be easily generalized. The techniques most commonly utilized are the following:

- Use Virtual GPUs (vGPUs), like NVIDIA MIG
- Use NVIDIA MPS to pre-allocate resources per process
- Create a shim layer between the process and the GPU
- Schedule workloads in time-slices or sequentially

The drawback of virtual GPUs is that off-the-shelf or older GPUs, such as RTX series cards, GTX 1080ti, or K80, lack support for virtualization technologies like NVIDIA MIG, limiting their ability to efficiently partition and schedule workloads in this way.

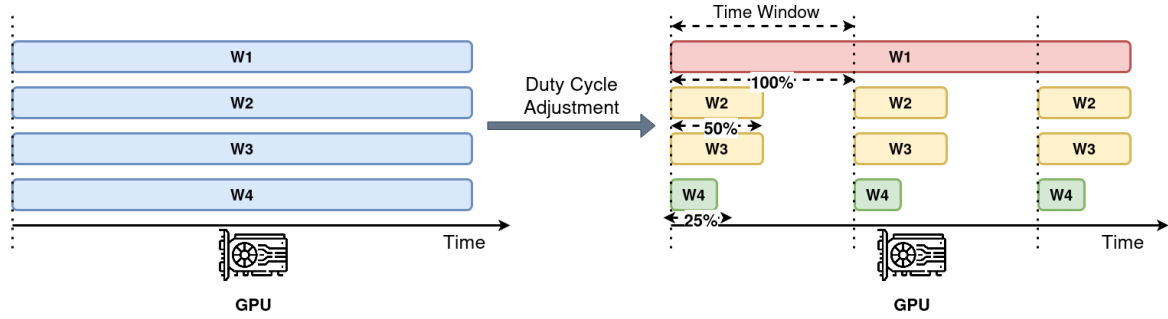


Fig. 2: Workload duty cycle adjustment for prioritizing workload execution in shared GPU environment

An alternative way to overcome the challenge of scheduling multiple processes on a shared GPU is to use NVIDIA MPS or the Multi-Process Service where the scheduler sets up an upper limit on the number of GPU threads that each workload can use in a shared GPU. The drawback of this method is that changes to the GPU thread allocation require restarting the processes, potentially necessitating complex checkpoint and restore techniques or even re-execution; this has the risk of losing all current progress.

A more sophisticated method proposed in the literature is the introduction of a shim layer between the processes and the GPU [13], [9]. The purpose of this approach is to orchestrate the execution of kernels on the GPU. Most of the approaches either propose a dynamically loaded library that overrides the CUDA kernel-specific methods (like `cudaLaunchKernel`) or make modifications on the GPU driver or the workload framework. Although this approach allows very fine tuning of the scheduling decisions at each GPU in a very low level, it has inherent difficulties. For example, the workload should use only dynamically linked CUDA libraries in order for the `LD_PRELOAD` to transpose the methods and use the specific methods that are transposed in the shim layer. Also modifications to the GPU driver can introduce compatibility issues, security issues, and software supply chain problems, as drivers should be periodically updated. Finally, modifications to the workload frameworks are not practical and typically are workload specific and cannot be integrated with other workloads.

B. Manipulating the GPU

Our goal is to provide a general purpose, high-level technique that allows heterogeneous workloads to satisfy their SLOs on shared GPUs, while having low overhead. As we previously discussed, the kernel launches originate from the process that is executed on the CPU; by manipulating the scheduling of this process on the CPU, we can directly manipulate the rate with which kernels are submitted to the GPU giving high priority processes more time to use the available GPU resources. To achieve that, our LCS Scheduler creates a custom *cgroup* for each process that uses the GPU. A *cgroup* (control group) is a Linux kernel feature that allows limiting and isolating resource usage like the CPU and the

memory of the processes. By *freezing* a *cgroup* the Linux scheduler pauses the execution of the processes in the *cgroup* on the CPU until it is *unfrozen*. Freezing and Unfreezing the *cgroup* can be easily achieved by writing to virtual files of the Linux kernel with very low overhead.

C. Execution Model

Each workload represents a batch job associated with a set of characteristics and requirements that remain constant throughout the execution of the job. Typically, these workloads are well-defined and repetitive, requiring defined GPU resources like memory and compute (e.g. Streaming Multiprocessors, Encoder / Decoder) and may have constraints related to their execution time (e.g. deadline). While the requirements and characteristics of a workload remain unchanged during a single execution, they may vary across different instances of the same workload. We consider each workload execution W to be characterized by a deadline or a pre-assigned priority level, and by a GPU memory requirement. The LCS natively supports two scheduling techniques: *Priority Based Scheduling* based on a set of pre-defined priority classes and *Deadline Based Scheduling* for workloads with real-time characteristics that are scheduled according to their SLOs (*i.e.*, deadline). The deadline, according to the application, can be either soft or hard. With a soft deadline we can leave the workload to execute even after the deadline time is elapsed, *i.e.*, if there are no other application waiting for execution. With a hard deadline the workload is evicted as soon as this deadline expires. We allow multiple workloads to run concurrently on the same GPU. In this case, the performance of each workload is affected by the other workloads that are co-located on the same GPU. Unfortunately there is not a straight forward way to calculate the degradation in performance of each workload as it depends on the characteristics and requirements of all the workloads that run simultaneously and the GPU scheduler.

D. Duty Cycle Adjustment

Our approach is inspired from the world of electronics and signal processing. The term duty cycle is a fundamental concept in systems involving periodic operations. It refers to the fraction of one complete cycle in which a system is active. Expressed as a percentage, the duty cycle quantifies how much

time a system remains "active" relative to the total period of the cycle. In our setup, the duty cycle refers to the fraction of the time (*activeDuration*) that a GPU enabled workload is allowed to launch kernels to the GPU during a cycle which we call *Time Window*. Figure 2 shows an example of four workloads with different priorities that run concurrently on a GPU. We observe that W1, which has the highest priority, fully utilizes the GPU during each epoch. The LCS can pause the execution of each workload based on its *activeDuration* by freezing the cgroup that is responsible for it. In the case that the workload needs full utilization of the GPU compute resources, lower priority workloads can be paused as long as needed.

A higher duty cycle indicates a longer *activeDuration*, while a lower duty cycle signifies a shorter *activeDuration*. This control mechanism allows the different workloads to share the GPU "proportionally" within a *Time Window*. Workloads with higher priority have increased duty cycle compared to workloads of lower priorities. This approach ensures that higher priority workloads execute in isolation during designated time intervals, during which lower-priority workloads remain idle, allowing them to use more resources of the GPU. In order to present the impact of the duty cycle adjustment on the performance of a workload, we designed an experiment with four different workloads sharing the same GPU. In Figure 3 we observe the latency per training step in μs of a Tensorflow job that runs concurrently with three other jobs on the same GPU, a NVIDIA RTX 4090. In the first part, we observe that the average latency is around $3700\mu s$. At this point all workloads are running concurrently without any limitations. At the second part we observe the training step latency of the workload while running in isolation, without the effect of the other three workloads, and finally in the third part we adjust the duty cycle of the three other workloads to be 50%, while maintaining the duty cycle of the "critical" workload at 100%. At this point we observe that the latency per training step of the "critical" workload fluctuates with a minimum value of $2500\mu s$ and a maximum value of $3700\mu s$. The *Time Window* was set to 1000ms, the active duration of the lower priority workloads was adjusted to 500ms due to the 50% duty cycle. Therefore, any training steps that were executed during this period of inactivity were executed in isolation experiencing no delays and any training steps that were executed during the period that the other workloads were active, slowed down.

Thus, by adjusting the duty cycle of the workloads we can provide a spatio-temporal sharing mechanism for scheduling different workloads on the same GPU.

E. Requesting shareable GPU

Our objective in this work is to develop a solution that requires minimal changes to the existing code bases. We have created a library that can be invoked each time the developer requests access to a shareable GPU. The library includes two functions *request_share()* and *release_share()*. *request_share()* is used when the developer requires access to a shareable GPU

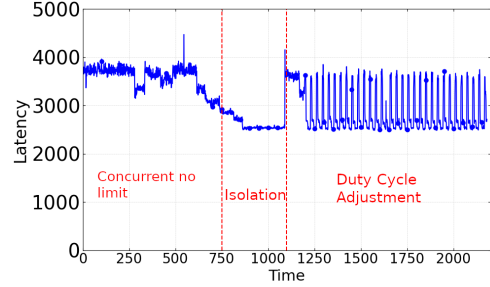


Fig. 3: Latency in μs per training step of a Tensorflow workload, with and without duty cycle adjustment while running concurrently with 3 other workloads as well as running in isolation on the same GPU.

of the host, for example, during neural network training or video encoding/decoding. The function automatically acquires the process id *pid* of the process that invokes it and informs the LCS that the process is requesting access to the GPU. Then the LCS scheduler adds the process to a new cgroup and assigns it a priority based on the scheduling algorithm. In the case that a fork is needed by the process, like when calling FFMpeg, then the developer has to create a wrapper around the new generated process and request share for the newly generated process, by providing its *pid*. In Fig. 4, we observe the two ways that a developer can manually request a share from a GPU. The red arrows indicate the necessary changes on the application code. If a change in the application code is not viable, or for applications that perform only GPU related operations like local ML-training, an environment variable *GPU_ONLY = TRUE* can be provided without any modifications to the application code. In this way, the LCS will automatically detect the process id and will immediately start the scheduling process for this application. However, this approach should only be used in such special cases, as it can cause delays to the execution of the application in regions where the GPU is not utilized like I/O.

F. Priority Based Scheduling

We have implemented two scheduling algorithms. The first algorithm, Priority Based Scheduling, is a scheduling algorithm that uses the predefined priorities given by the users for each workload to determine the scheduling order of the workloads. Each workload can be assigned one of a set of predefined priority classes such as *HIGH*, *MEDIUM*, *LOW*. The number of priority classes is a tunable parameter that the user can define as he wishes. Each priority is associated with a different value for the duty cycle. In this way higher priority workloads receive more time or attention, akin to a duty cycle's on-off modulation and ensure that critical tasks are handled more efficiently. This technique has a lot in common with older MPS techniques provided by NVIDIA, the difference with our approach is that it can change the priority of each workload dynamically without the need to

Workload Id	Description	GPU Load (RTX4090)	Priority Scenario 1 (RTX4090)	Priority Scenario 2 (RTX4090)	Priority Scenario 3 (GTX1060)
CNN-lg	Large CNN Model for CIFAR-10	51%	High/Real-Time	Low	High/Real-Time
DNN-md	Medium Size Deep Neural Network for Higgs dataset	96%	Medium	High/Real-Time	-
DNN-lg	Large Size Deep Neural Network for Higgs dataset	98%	Low	Medium	-
CNN-md	Medium CNN Model for CIFAR-10	17%	High/Real-Time	-	Medium
ENC	Video Encoding with FFMpeg using NVENC	27%	High/Real-Time	-	-
DNN-md2	Medium Size Deep Neural Network for Higgs dataset	96%	Low	Medium	Low

TABLE I: Description of the workloads used in evaluation

restart the workload. Details of the algorithm can be seen on Algorithm 1.

The algorithm works as follows: it divides the workloads into N distinct categories (3 in our example) based on the priority that the user gives during the submission of the workload. Then periodically, every n ms, the algorithm counts the number of active processes in each category. n is a tunable parameter of the system. The *DUTY_CYCLE* of lower priority workloads can be adjusted according to the number of workloads on the above priority tiers. For example, if no *HIGH* priority workloads are found, *MEDIUM* priorities increase their *DUTY_CYCLE* to *HIGH*. If a new *HIGH* priority workload is submitted, then the *MEDIUM* workloads will decrease their *DUTY_CYCLE* to their normal value. In this way, lower priority workloads can utilize the full compute capacity of the GPU if no other more critical workloads run, maintaining the GPU utilization high.

Priority based scheduling is the simplest way to schedule heterogeneous workloads on the same GPU and does not require progress update notifications to the RTPM. Therefore it is compatible with any workload without needing modifications to its source code.

G. Laxity Based Scheduling for Real-time workloads

The second scheduling algorithm we have developed is Least Laxity scheduling (LLS), a dynamic real-time scheduling algorithm that dynamically prioritizes tasks based on their laxity, also known as slack time. For each real-time workload W , the RTPM calculates the laxity L based on its reported progress, the deadline and its estimated remaining time. As laxity L we denote the time that the workload can spare without violating its deadline. More formally, each workload W_i is characterized at a specific time t by D_i which is the absolute deadline of workload, t which represents the current time and $C_i(t)$ which is the remaining computation time of W . The laxity $L_i(t)$ or workload W_i at time t is defined as:

$$L_i(t) = D_i - t - C_i(t) \quad (1)$$

The value of $C_i(t)$ is estimated by the RTPM module by the metrics that it computes periodically assuming a linear progress increase over time. As the RTPM estimates this value periodically, it automatically re-adjusts it in case of changes, so the linearity holds inside each time window.

Algorithm 1 Fix Priorities

```

1: numberOfHigh  $\leftarrow$  size of StaticFields.highPriority
2: numberOfMedium  $\leftarrow$  size of StaticFields.mediumPriority
3: medPr  $\leftarrow$  MED_DUTYCYCLE
4: lowPr  $\leftarrow$  LOW_DUTYCYCLE
5: if numberOfHigh = 0 then
6:   medPr  $\leftarrow$  HIGH_DUTYCYCLE
7:   lowPr  $\leftarrow$  MED_DUTYCYCLE
8:   Print "High priority == 0, Medium  $\rightarrow$  High"
9: end if
10: if numberOfMedium = 0  $\wedge$  numberOfHigh = 0 then
11:   lowPr  $\leftarrow$  HIGH_DUTYCYCLE
12:   Print "Medium priority == 0, High priority == 0, Low  $\rightarrow$  High"
13: else if numberOfMedium = 0  $\wedge$  numberOfHigh > 0 then
14:   lowPr  $\leftarrow$  MED_DUTYCYCLE
15:   Print "Medium priority == 0, Low  $\rightarrow$  Medium"
16: end if
17: for all entries in mediumPriority do
18:   Set entry's process priority to medPr
19: end for
20: for all entries in lowPriority do
21:   Set entry's process priority to lowPr
22: end for

```

Laxity is an indication of urgency. When the laxity of a workload is low the probability of this workload to miss its deadline increases. If the laxity becomes negative, the workload is guaranteed to miss its deadline if no actions are made. Laxity-based scheduling is a well-known technique that helps to minimize resource contention and ensures that real-time workloads meet their deadlines while non real-time workloads can be delayed without significant impact. Laxity-based scheduling aims to optimize overall system performance while maintaining fairness in a shared environment like shared GPUs.

As we previously explained in detail, laxity is an indication of urgency for a workload. Laxity can be very useful for scheduling workloads with real-time characteristics such as neural learning over streaming data [14], where the result of the training is used to directly update the weights of neural networks that currently are in use. In contrast with priority

```

def train(model,n_epochs):
    → request_share(gpu)#requesting shareable GPU
    for epoch in range(n_epochs):
        train(model, device, dataloader, optimizer, epoch)
        scheduler.step()
    → release_share(gpu)#releasing shareable GPU

def encodeVideo(file):
    downloadVideo(file)
    proc = subprocess.Popen(["ffmpeg",...])
    #requesting shareable GPU for the spawned process
    → request_share(gpu,proc.pid)
    code = proc.wait()
    → release_share(gpu,proc.pid)

```

Fig. 4: Code modification for requesting a shareable GPU

Algorithm 2 Priority and Real-time scheduling algorithm

```

1: while true do
2:   restoreDutyCyclesIfNoCritical() {Restore priorities to
   normal if no critical processes exist}
3:   for all process proc in processes do
4:     if proc.getState() = COMPLETED then
5:       processes.remove(proc)
6:     else
7:       if proc.priority = critical then
8:         if try = 0 then
9:           {The first time adjust the duty cycle to the
           worst case}
10:          deadline ← proc.deadline - DE-
            SIREL_LAXITY
11:          dutyCycleProc ←  $\frac{100 \cdot \text{proc.isolation}}{100 - \text{dutyCycleProc}}$ 
12:          dutyCycleOthers ←  $\frac{\text{deadline}}{100}$ 
13:          activeDuration ←  $\lfloor 1000 \cdot \text{dutyCycle} \rfloor$ 
14:          try ← try + 1
15:          lastChange ← now()
16:        else
17:          if laxity ≥ UPPER_LIM and now() -
            lastChange ≥ 5s then
18:            activeDuration ← activeDuration + 50ms
19:            try ← try + 1
20:            lastChange ← now()
21:          else if laxity ≤ LOWER_LIM and now() -
            lastChange ≥ 5s then
22:            activeDuration ← activeDuration - 50ms
23:            try ← try + 1
24:            lastChange ← now()
25:          end if
26:        end if
27:      end if
28:    end if
29:  end for
30: end while

```

based scheduling where there is no need to provide real-time feedback to the scheduler about the progress of each workload, with laxity-based scheduling we need to provide information about the progress of the workload in real time. In this way, the scheduler, and especially the RTPM, can predict the execution time of the workload and as a result its laxity. For each workload with real-time characteristics, we can assign a target laxity value during the registration of the workload. In this

way, we can ensure that the workload will complete before its deadline. For this reason we have to ensure that the real-time or critical workloads have sufficient resources in order to complete in time. We can adjust the *DUTY_CYCLE* of the other workloads that run concurrently on the same GPU, to leave more resources available for the critical workloads. By estimating the execution time of each real-time workload in isolation, we can calculate the worst-case execution time in a shared GPU. We know that

$$DC_{isolation} \cdot T_{isolation} = DC' \cdot T' \quad (2)$$

where $DC_{isolation}$ is the duty cycle of the workload when it runs in isolation, which is 100% and $T_{isolation}$ is the time it takes, which can be estimated by historical data or profile runs. T' is the deadline that we want the workload to achieve, which is a given parameter by the user and DC' is the minimum *DUTY_CYCLE* that the workload needs to use in order to achieve the desired deadline. We have to notice here that $T' \geq T_{isolation}$. Rearranging for DC' we have

$$DC' = \frac{DC_{isolation} \times T_{isolation}}{T'} \quad (3)$$

This *DUTY_CYCLE* DC' is the time duration within each Time Window that the workload needs to have exclusive access to the GPU resources. So, workloads running concurrently at the same GPU need to adjust their *DUTY_CYCLE* period to $DC_{others} = 100\% - DC'$. However, the workload can utilize a portion of the GPU resources concurrently with the other workloads within each time slice. In this way, the time that the workload will take to complete can be significantly less than the estimated. For this reason we have developed a real-time algorithm that monitors the *laxity* of the real-time workload and automatically fine-tunes the duty cycle of the non-critical workloads in order to keep the laxity in the required range. Algorithm 2 shows the actual steps that we perform in order to keep the laxity inside the desired range.

In the case that multiple real-time workloads run concurrently, we select the highest *DUTY_CYCLE* that allows the critical workloads to complete on time without violating their SLOs.

V. EVALUATION

In order to evaluate the full potential of our approach, we have designed a set of experiments based on three different scenarios of GPU sharing between a set of workloads of various types, frameworks, like PyTorch, Tensorflow and FFMpeg, and GPU load. For each workload we assign a priority {Low,

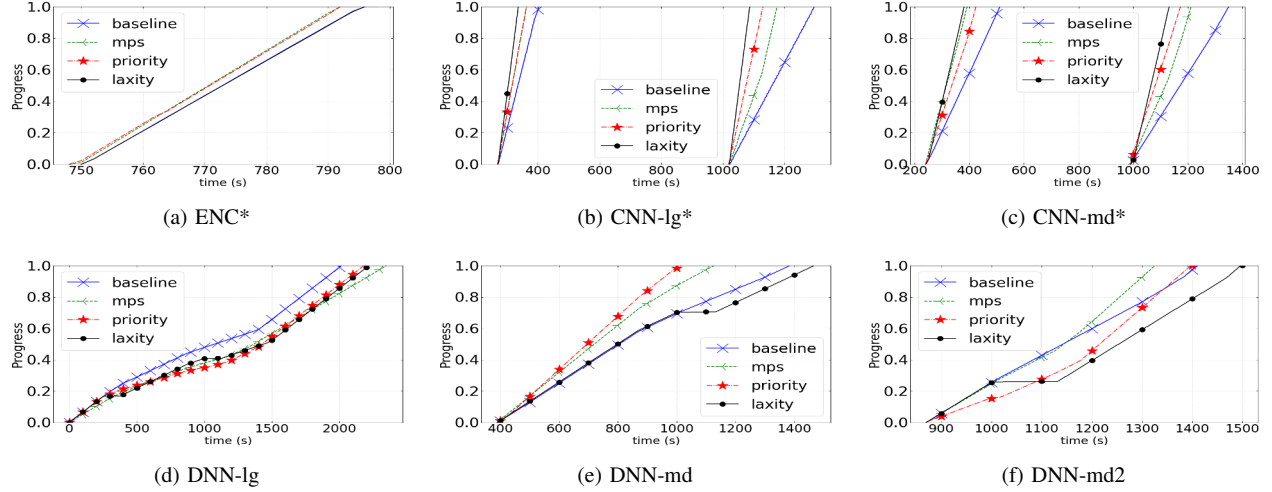


Fig. 5: Progress over time (seconds) for workloads in Scenario #1 with different scheduling approaches. Workloads with * indicate *High* priority / Real-time requirements (RTX4090)

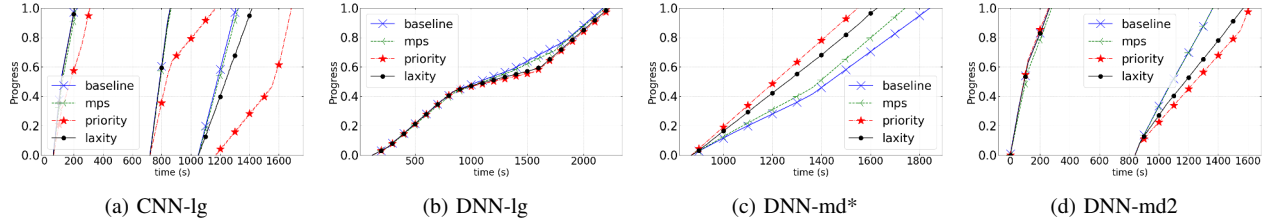


Fig. 6: Progress over time (seconds) for workloads in Scenario #2 with different scheduling approaches. Workloads with * indicate *High* priority / Real-time requirements (RTX4090)

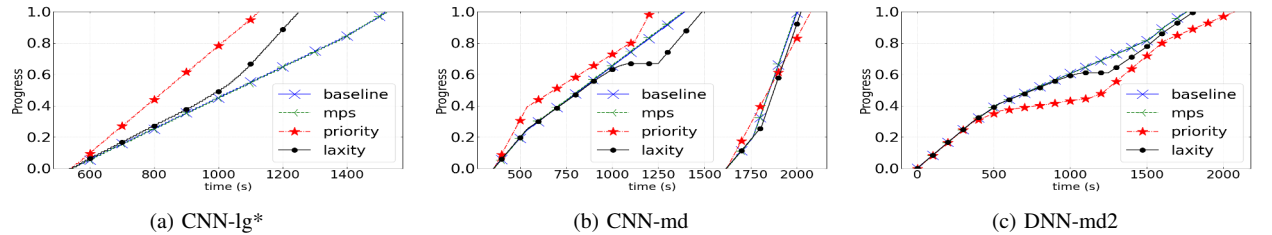


Fig. 7: Progress over time (seconds) for workloads in Scenario #3 with different scheduling approaches. Workloads with * indicate *High* priority / Real-time requirements (GTX1060)

Medium, High/Real Time} for each one of the three scenarios. Details about the different workloads can be found on table I.

A. Evaluation Environment

We evaluated our approaches on two machines with NVIDIA GPUs of different generations and architectures to illustrate the generality of our approach. The first node is equipped with an Intel i9-14000kf clocked at 5.5 GHz with disabled e-cores for performance stability, 64GB of RAM and one NVIDIA GeForce RTX 4090 with 24GB memory (Ada Lovelace architecture). It runs Ubuntu Server 22.04LTS with linux kernel 6.8. The second node has an Intel i7-12700 with

64GB of RAM and one NVIDIA GeForce GTX 1060 with 3GB of memory (Pascal architecture). It runs Ubuntu Server 22.04LTS with linux kernel 5.15. On both nodes the LCS component and an NVIDIA MPS server were running. Both nodes have installed the NVIDIA 550 driver and CUDA 12.04.

B. Scenarios

We evaluated our approach under three different scenarios. The first two scenarios were deployed on the node with the RTX4090 and the third scenario was deployed on the node with the GTX1060. The aim of our experimental evaluation is to provide meaningful insights into GPU scheduling tech-

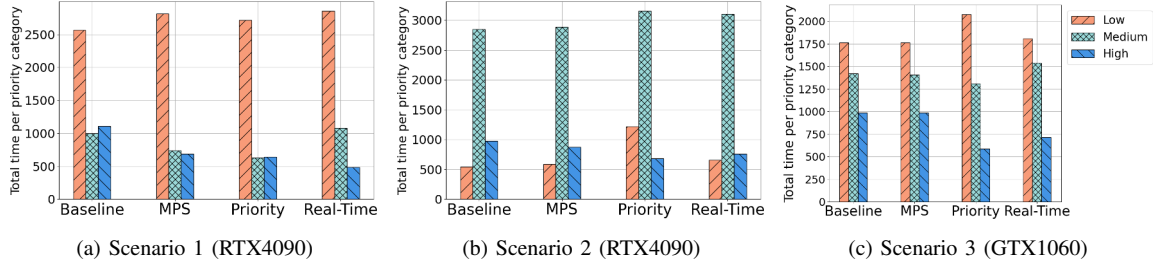


Fig. 8: Total time in seconds for all workloads grouped by priority for the three different scenarios.

niques for standalone cluster nodes. Our focus on single-node environments allows us to analyze and investigate the behavior of different GPU scheduling strategies. To ensure that our findings are not limited to a specific hardware configuration and can be generalized across a range of systems, we deliberately selected GPUs from two different hardware generations. The selection of two different architectures allows us to assess the performance and adaptability of our proposed approaches under diverse computational characteristics and architectural constraints.

In each scenario there was overlap between the execution of the different workloads in order to prove the effectiveness of our proposed scheduling strategies. Scenario one has more than one real-time workloads with high priority, which is indicated with star (*) in the legends of the charts. The priorities of the workloads for each scenario can be seen in table I. We compared our proposed strategies, Priority Based Scheduling and Real-time Scheduling against: (i) Baseline strategy, where each workload runs with MPS on the node, but without any limitations. (ii) An MPS Priority strategy where each workload runs again with MPS but this time has a priority defined with *CUDA_MPS_ACTIVE_THREAD_PERCENTAGE*. High priority workloads are set to use 100% of the Threads, Medium priority are set to use 50% and Low priority 25%.

In Figures 5,6,7 we observe the progress over time of each workload under the three different scenarios with the four different scheduling strategies. X-axis represents the duration of the experiment in seconds and Y-axis shows the progress of the workload at every moment. The beginning of each line indicates the moment that the workload is submitted, and the end of each line indicates the moment that the workload finished. The gradient of the line represents the rate that the progress of each workload changes over time. Steeper gradient indicates an accelerated workload, and as the gradient declines, this indicates that the workload suffers latencies. Flat lines indicate that the workload have been paused; this can happen each time the *DUTY_CYCLE* of the workload reaches zero. As we explained, the gradient of the progress of each workload indicates the speedup or slow-down of the workload. The decline in the gradient is caused either by the increased load on the GPU (Baseline, MPS) or by the *DUTY_CYCLE* adjustment in Priority Based scheduling or Real-time scheduling. We observe that the Real-

time scheduling performs adjustments only when the laxity of the real-time workloads falls under the specified range, indicating a possible deadline miss. In Figure 7 we observe that the MPS and Baseline approaches are identical, this is caused by the fact that GTX 1060 is a pre-Volta generation GPU and *CUDA_MPS_ACTIVE_THREAD_PERCENTAGE* is not available. However we included this approach for completeness of our evaluation. Across all Figures [5,6,7] we observe that both Priority based and Real-time scheduling allowed the real-time or high priority workloads to finish earlier than the Baseline or MPS approaches. In the first scenario we selected a very strict deadline value for real-time workloads in order to prove that the real-time scheduler can indeed achieve the deadline, even if it is necessary to adjust the *DUTY_CYCLE* of the medium and low priority workloads to a lower level. In the other two approaches, we chose more relaxed deadlines, 80% longer than the time it takes in isolation on each card. The Real-time scheduler acts more aggressively on lower priority workloads compared to the Priority based scheduler in cases where it detects a possible deadline-miss. This is indicated by flattened gradients in the Figures [5e,5f,7b,7c]. While the Priority based scheduler dynamically adjusts the *DUTY_CYCLE* of each workload to allow more time for the high priority workloads to exclusively use the GPU, the Real-Time scheduler automatically adjusts the *DUTY_CYCLE* of lower priority workloads in order to allow the high priority / real-time workload to achieve its SLO. Therefore the adjustments may be more aggressive compared to the Priority based scheduler in scenarios with strict deadlines.

In Figure 8 we indicate the total time that the workloads ran during the duration of the experiments grouped by priority. We can observe that priority based scheduling penalizes more the low priority workloads compared to the medium in order to accelerate the high priority workloads. On the other hand, real-time scheduling equally penalizes low and medium workloads in order to achieve the deadlines of the real-time workloads. In scenario 1 we observe that the total duration of the *High priority* workloads is around 480s with the Real-time scheduler, 680s with the MPS and 1100s in the Baseline. The Real-time scheduler allows the real-time workloads to achieve 30% lower total latency compared to NVIDIA MPS and 56% lower total latency compared to Baseline translated

to $\approx 1.41\times$ and $\approx 2.29\times$ speedup. In all scenarios the high priority/real-time workloads performed better compared to the NVIDIA MPS approach without excessive slowdown in the low and medium workloads. Finally, in order to measure the overhead of our solution, we conducted an experiment where we concurrently executed two PyTorch workloads that each one takes 61s in isolation to complete and 64s when running concurrently. We adjusted the *DUTY_CYCLE* of the first workload to 50% and we measured their duration again when running concurrently. The duration of the adjusted workload measured at 128.5s which indicates that the overall overhead is negligible. We have to note here, that, this overhead does not affect the real-time and high priority workloads.

The system is designed with the flexibility to support both monolithic and distributed workloads, enabling it to accommodate a wide range of workloads. In our experimental evaluation, we focused on monolithic workloads, like DNN training and video encoding tasks, which are executed on single nodes without requiring coordination across multiple devices. However, the system can be seamlessly used in distributed workloads, such as multi-node or multi-GPU distributed training jobs. In these scenarios, the user is required to assign a priority level or deadline in the job and the system will guarantee that these parameters are consistent across all distributed processes that constitute the distributed job. The system dynamically adjusts the duty cycles of lower-priority tasks that share GPU resources with higher-priority workloads in each device and can ensure that the desired SLOs for the whole job will be satisfied.

VI. RELATED WORK

Garg et. al. in Share-a-GPU [15], introduce a cooperative multitasking approach to provide simple and effective time-sharing on GPUs for GPGPU programs written in CUDA. The core idea is to allow multiple workloads to run concurrently by giving each exclusive access to the GPU(s) in a round-robin manner over a time quantum. Although their approach share many concepts with our technique regarding exclusive access to the GPU for high priority workloads over a time quantum, there are significant differences. First of all, their approach is relatively old and developed at a period when concurrent execution on a GPU was not possible. Also their approach acts at a lower level compared with ours and does not provide a read- to-use solution for ML frameworks such as PyTorch and Tensorflow.

Hsieh et. al. in Voda [16] propose a GPU scheduling platform for elastic deep learning specifically designed for Kubernetes clusters. Their main idea is to provide an efficient and easy-to-use platform that allows various scheduling algorithms for elastic machine learning training. Their approach emphasizes on the machine learning workload scheduling on a set of cluster nodes, without considering the scheduling on shareable GPUs. On the other hand, our solution emphasizes on the scheduling of heterogeneous workloads, and not only machine learning training, on shareable GPUs and can be

included as a part of a cluster wide workload scheduler like Voda.

In [17], the authors propose dynamic resource allocation techniques to optimize performance, particularly for applications with linear dependencies between components. The primary contributions are the use of shared GPU memory between components to eliminate unnecessary data copies between the CPU and GPU, significantly reducing memory transfer overhead resulting in improved performance compared to MPS approach.

Orion [9] is a system designed for interference-aware, fine-grained GPU sharing specifically for Machine Learning (ML) applications, addressing the problem that Deep Neural Network (DNN) workloads often underutilize GPUs. Orion operates by transparently intercepting GPU kernel launches from multiple clients sharing a GPU and scheduling work at the granularity of individual operators. Its scheduling policy considers factors such as client job priority, operator size, and compute vs. memory intensity. Although it can provide very fine-tuned scheduling, Orion is only specialized in machine learning jobs on NVIDIA GPUs. Also it requires dynamically linked NVIDIA libraries in order to intercept the kernel launches.

DynamoML [18] is a system designed for dynamic resource management of mixed Machine Learning (ML) workloads (modeling, training, and inference jobs) on GPU clusters, addressing the limitations of existing resource managers that are often application-oblivious or static. It provides a fine-grained shared GPU allocator, an application-aware distributed training job scheduler, and a performance-driven inference service auto-scaling controller. Similarly to Orion, DynamoML is only machine learning oriented and requires the interception of kernel launches with LD_PRELOAD.

There is rich literature on High Performance Computing (HPC) frameworks that aim to schedule tasks on heterogeneous resources like CPUs, GPUs and FPGAs. ParSEC [19], [20], [21] is a runtime framework for efficient execution of microtasks on distributed, heterogeneous, many core systems. Applications are modeled as directed acyclic graphs (DAGs), where tasks are nodes and edges represent data dependencies. These DAGs are stored in a compact format that allows efficient, distributed, and scalable querying of data dependencies. StarPU [22] is a unified runtime system that supports task-based programming on heterogeneous multicore architectures. It abstracts hardware details and automatically schedules tasks across CPUs, GPUs, and other accelerators. OmpSs [23] is a task-based programming model that extends OpenMP with new directives to support asynchronous parallelism and heterogeneity. OmpSs schedules tasks dynamically across heterogeneous resources such as CPUs, GPUs, or FPGAs. This approach simplifies parallel programming while enabling scalability and efficient hardware utilization.

These frameworks introduce a programming model that can be used to create complex HPC applications and then schedule them to dedicated resources. In contrast, our work focuses on off-the-shelf GPU enabled applications requested

by users, ranging from DNN training jobs to video encoding. Our aim is to provide hardware-agnostic GPU spatio-temporal sharing and scheduling mechanisms for GPU intensive, long-running jobs in shared edge clusters requiring non-to-minimum changes in the workload source code. In our work, we have implemented a thin scheduling layer that acts in the kernel space (cgroups), which is compatible with different GPU architectures and works in concert with GPU scheduling components such as NVIDIA MPS in order to dynamically schedule tasks based on the corresponding priorities.

Furthermore, HPC frameworks typically consider applications as a directed acyclic graph (DAG) of tasks with data dependencies on the DAG vertices. In the current state of our work, we consider applications either as monolithic or as a set of replicated tasks for increased performance, like distributed ML training.

VII. CONCLUSION

In this paper we presented our approach of allowing multiple GPU enabled workloads to dynamically share GPUs that do not support virtualization. LCS allows heterogeneous workloads to dynamically allocate a portion of the GPU resources inside time-sliced windows while maintaining the concurrent execution of the workloads without introducing additional overhead and requiring minimal to no changes on the workload code base. Our evaluation showed that LCS can adapt to various scheduling like priority based policy while guaranteeing the SLO requirements of real-time workloads. In our evaluation we measured our solution to achieve up to 56% lower latency compared to baseline and 30% compared to MPS. In our future work we plan to focus on supporting pipelines of GPU enabled applications and scheduling workloads with dynamic resource requirements.

ACKNOWLEDGMENT

The research work was supported by the Hellenic Foundation for Research and Innovation (HFRI) under the 3rd Call for HFRI PhD Fellowships (Fellowship Number: 6812), by the European Union through the EU ICT-48 2020 project TAILOR (No. 952215), the H2020 AutoFair project (No. 101070568), the Horizon Europe CoDiet project (No. 101084642) and the HORIZON-WIDERA-2023-ACCESS-02-01 ETHCSTWIN project (No. 101159928).

REFERENCES

- [1] (2025) Nvidia mps. Accessed: 2025-05-01. [Online]. Available: <https://docs.nvidia.com/deploy/mps/index.html>
- [2] (2025) Nvidia time-slicing kubernetes. Accessed: 2025-05-01. [Online]. Available: <https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/latest/gpu-sharing.html>
- [3] (2025) Nvidia mig. Accessed: 2025-05-01. [Online]. Available: <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
- [4] (2025) Tensorflow. Accessed: 2025-05-01. [Online]. Available: <https://www.tensorflow.org/>
- [5] (2025) Pytorch. Accessed: 2025-05-01. [Online]. Available: <https://pytorch.org/>
- [6] (2025) Nvidia nvml. Accessed: 2025-05-01. [Online]. Available: <https://developer.nvidia.com/management-library-nvml>
- [7] (2025) Prometheus. Accessed: 2025-05-01. [Online]. Available: <https://prometheus.io/>
- [8] D. M. Naranjo, S. Risco, C. de Alfonso, A. Pérez, I. Blanquer, and G. Moltó, "Accelerated serverless computing based on gpu virtualization," *Journal of Parallel and Distributed Computing*, vol. 139, pp. 32–42, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731519303533>
- [9] F. Strati, X. Ma, and A. Klimovic, "Orion: Interference-aware, fine-grained gpu sharing for ml applications," in *Proceedings of the Nineteenth European Conference on Computer Systems*, ser. EuroSys '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1075–1092. [Online]. Available: <https://doi.org/10.1145/3627703.3629578>
- [10] J. Gu, Y. Zhu, P. Wang, M. Chadha, and M. Gerndt, "Fast-gshare: Enabling efficient spatio-temporal gpu sharing in serverless computing for deep learning inference," in *Proceedings of the 52nd International Conference on Parallel Processing*, ser. ICPP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 635–644. [Online]. Available: <https://doi.org/10.1145/3605573.3605638>
- [11] J. Gu, S. Song, Y. Li, and H. Luo, "Gaiagpu: Sharing gpus in container clouds," in *2018 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Ubiquitous Computing & Communications, Big Data & Cloud Computing, Social Computing & Networking, Sustainable Computing & Communications (ISPA/IUCC/BDCLOUD/SocialCom/SustainCom)*, 2018, pp. 469–476.
- [12] B. Wu, Z. Zhang, Z. Bai, X. Liu, and X. Jin, "Transparent GPU sharing in container clouds for deep learning workloads," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 69–85. [Online]. Available: <https://www.usenix.org/conference/nsdi23/presentation/wu>
- [13] C. Lv, X. Shi, Z. Lei, J. Huang, W. Tan, X. Zheng, and X. Zhao, "Dilu: Enabling gpu resourcing-on-demand for serverless dl serving via introspective elasticity," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 311–325. [Online]. Available: <https://doi.org/10.1145/3669940.3707251>
- [14] G. Klioumis and N. Giatrakos, "Data-driven synchronization protocols for data-parallel neural learning over streaming data," in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 988–997.
- [15] S. Garg, K. Kothapalli, and S. Purini, "Share-a-gpu: Providing simple and effective time-sharing on gpus," in *2018 IEEE 25th International Conference on High Performance Computing (HiPC)*, 2018, pp. 294–303.
- [16] T.-T. Hsieh and C.-R. Lee, "Voda: A gpu scheduling platform for elastic deep learning in kubernetes clusters," in *2023 IEEE International Conference on Cloud Engineering (IC2E)*, 2023, pp. 131–140.
- [17] H. Sedighi, D. Gehberger, and R. Glioth, "Workload-aware dynamic gpu resource management in component-based applications," in *2022 IEEE International Conference on Cloud Engineering (IC2E)*, 2022, pp. 213–220.
- [18] M. Chiang and J. Chou, "Dynamoml: Dynamic resource management operators for machine learning workloads," in *Proceedings of the 11th International Conference on Cloud Computing and Services Science - CLOSER, INSTICC*. SciTePress, 2021, pp. 122–132.
- [19] (2025) Parsec. Accessed: 2025-05-01. [Online]. Available: <https://icl.utk.edu/parsec/>
- [20] G. Bosilca, A. Bouteiller, A. Danalis, T. Herault, P. Lemariner, and J. Dongarra, "Dague: A generic distributed dag engine for high performance computing," Anchorage, Alaska, USA: IEEE, 2011-00 2011, pp. 1151–1158.
- [21] A. Bouteiller, T. Herault, Q. Cao, J. Schuchart, and G. Bosilca, "Parsec: Scalability, flexibility, and hybrid architecture support for task-based applications in ecp," *The International Journal of High Performance Computing Applications*, 2024-10 2024. [Online]. Available: <https://journals.sagepub.com/doi/10.1177/10943420241290520>
- [22] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, "StarPU: a unified platform for task scheduling on heterogeneous multicore architectures," *Concurrency and Computation: Practice and Experience*, vol. 23, no. 2, pp. 187–198, 2011. [Online]. Available: <https://inria.hal.science/inria-00550877>
- [23] (2025) The ompss programming model. Accessed: 2025-05-01. [Online]. Available: <https://pm.bsc.es/ompss>